

✓ CalendarMate Workflow

CalendarMate Workflow — Logic

1. User Request Arrives A user asks something — “What’s my day?”, “Schedule a meeting”, “Summarize emails”, “Do I have conflicts?”, “What were yesterday’s action items?”

This raw text goes straight into the Intent Classifier.

2. Intent Classifier Decides the Category The classifier chooses exactly one intent:

Briefing

Scheduling

Email Summary

Conflict Check

Follow-Up

It never guesses missing details and always returns valid JSON.

This step determines which agent should handle the request.

3. Orchestrator Routes the Request Based on the detected intent, the orchestrator sends the request to the correct agent:

Briefing Agent

Scheduling Agent

Email Summary Agent

Conflict Check Agent

Follow-Up Agent

This routing is deterministic — one request → one agent.

4. Agent Performs Its Specialized Task Each agent has strict rules:

Briefing Agent

Turns calendar events into a natural-language daily summary.

Scheduling Agent

Extracts date, time, duration, attendees, title. If anything is missing, it asks a clarification question instead of guessing.

Email Summary Agent

Summarizes only the emails provided and highlights urgent items.

Conflict Check Agent

Compares the requested meeting time with existing events and reports overlaps.

Follow-Up Agent

Summarizes yesterday’s meetings and extracts action items.

Each agent returns a single JSON response.

5. Output Is Cleaned and Validated The system ensures:

JSON is valid

No markdown fences

No hallucinated details

No invented attendees or times

No scheduling unless explicitly asked

```
import json
from datetime import datetime, timedelta
from openai import OpenAI
import os

# Set the OpenAI API key from the provided context
os.environ["OPENAI_API_KEY"] = "sk-proj-SINcukg84SKtJN050sQRYJmkZsjdcaBcFIjWqKrUSdNaRC2VHM5PSyyHRPOFaJDup9GHef1T5zT3B1bkFJmPE7B"
```

```

client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))

# -----
# GLOBAL RULES
# -----
GLOBAL_SYSTEM = """
You are CalendarMate, an AI assistant that manages time, meetings, and email context.

GLOBAL RULES:
1. Always respond in valid JSON with a top-level field "response".
2. Never hallucinate missing details.
3. Never invent attendees, times, dates, or email content.
4. Never create events unless explicitly asked.
5. Never double-book unless user says override.
6. If information is missing, ask for clarification.
7. Keep responses concise.
8. Stop after producing JSON.
"""

# -----
# INTENT CLASSIFIER
# -----
def classify_intent(user_input):
    system = GLOBAL_SYSTEM + """
You are the Intent Classifier for CalendarMate.

Your job:
- Read the user request.
- Choose EXACTLY ONE of these intents:
    - Briefing → questions about “my day”, “my schedule”, “what’s happening”, “summary of meetings”
    - Scheduling → requests to schedule, book, find time, set up, arrange meetings
    - Email Summary → requests to summarize unread emails, urgent emails, emails needing attention
    - Conflict Check → questions about conflicts, overlaps, double-booking, “same time as”
    - Follow-Up → questions about yesterday’s meetings, action items, follow-ups

Rules:
- DO NOT guess based on keywords alone.
- DO NOT classify “summary of meetings” as Email Summary – that is Briefing.
- “Schedule”, “book”, “find time”, “set up” → ALWAYS Scheduling.
- “Conflicts”, “overlap”, “same time as” → ALWAYS Conflict Check.
- “Unread emails”, “emails needing attention”, “urgent emails” → ALWAYS Email Summary.
- “Yesterday”, “follow-up”, “action items” → ALWAYS Follow-Up.

Output:
Return ONLY valid JSON:
{"response": "<IntentName>"}
"""

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system},
            {"role": "user", "content": user_input}
        ]
    )
    return response.choices[0].message.content.strip()

# -----
# AGENT: BRIEFING
# -----
def briefing_agent(events):
    system = GLOBAL_SYSTEM + """
You are the Briefing Agent.

Your job:
- Produce a natural-language daily briefing.
- The output MUST be valid JSON with a top-level field "response".
- The value of "response" MUST be a human-readable summary, NOT raw JSON.
- Do NOT repeat or echo the input events JSON.
- Do NOT output lists of events.
- Convert each event into a readable sentence with time + title.
- If no events exist, return: {"response": "You have no events for this day."}
"""

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[

```

```

        {"role": "system", "content": system},
        {"role": "user", "content": json.dumps(events)}
    ]
)
return response.choices[0].message.content

# -----
# AGENT: SCHEDULING
# -----
def scheduling_agent(user_input):
    system = GLOBAL_SYSTEM + """
You are the Scheduling Agent for CalendarMate.

Your job:
Extract ONLY the following fields from the user request:
- date
- time
- duration
- attendees
- meeting title (if provided)

Rules:
1. NEVER invent missing details.
2. If ANY required detail is missing, ask a clarification question.
3. If attendees are missing → ask: "Who should I schedule this with?"
4. If date is missing → ask: "What date should this be scheduled for?"
5. If time is missing → ask: "What time should this be scheduled for?"
6. If duration is missing → ask: "How long should the meeting be?"
7. If meeting title is missing → ask: "What should the meeting be called?"
8. NEVER schedule automatically – only extract details or ask questions.
9. Output MUST be valid JSON with a top-level field 'response'.

Examples:

If details are complete:
{
  "response": {
    "date": "2026-05-19",
    "time": "14:00",
    "duration": "30 minutes",
    "attendees": ["sarah@company.com"],
    "title": "Meeting"
  }
}

If details are missing:
{
  "response": "Who should I schedule this with?"
}
"""
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system},
            {"role": "user", "content": user_input}
        ]
    )
    return response.choices[0].message.content

# -----
# AGENT: EMAIL SUMMARY
# -----
def email_summary_agent(emails):
    system = GLOBAL_SYSTEM + """
You are the Email Summary Agent.
Summarize ONLY the emails provided.
Highlight urgent/action items.
Ignore newsletters unless asked.
"""
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system},
            {"role": "user", "content": json.dumps(emails)}
        ]
    )

```

```

    )
    return response.choices[0].message.content

# -----
# AGENT: CONFLICT CHECK
# -----
def conflict_check_agent(user_input, events):
    system = GLOBAL_SYSTEM + """
You are the Conflict Check Agent for CalendarMate.

Your job:
- Determine whether the user's requested meeting time conflicts with any existing events.
- You will ALWAYS receive the user request AND the list of events.
- You must detect conflicts even when the user refers to an event indirectly
  (e.g., "my standup", "my sync", "my meeting at that time").

Rules:
1. If the user mentions an event by name (e.g., "standup"), match it to the existing events list.
2. If the user says "same time as my existing standup", infer the standup's time from the events list.
3. If the inferred time overlaps with any event → return a conflict.
4. If no conflict exists → return "No conflicts found."
5. NEVER invent events or times.
6. Output MUST be valid JSON with a top-level field "response".

Examples:

If conflict exists:
{
  "response": "You have a conflict at 10:00 AM with 'Team Sync'."
}

If no conflict:
{
  "response": "No conflicts found."
}
"""
    payload = {
        "request": user_input,
        "events": events
    }

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system},
            {"role": "user", "content": json.dumps(payload)}
        ]
    )
    return response.choices[0].message.content

# -----
# AGENT: FOLLOW-UP
# -----
def follow_up_agent(events):
    system = GLOBAL_SYSTEM + """
You are the Follow-Up Agent.
Summarize yesterday's meetings and extract action items.
"""
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system},
            {"role": "user", "content": json.dumps(events)}
        ]
    )
    return response.choices[0].message.content

# -----
# TEST HARNESS (T1-T12)
# -----
TEST_INPUTS = [
    "What does my day look like today?",
    "Give me a summary of my meetings for tomorrow.",
    "Schedule a 30-minute meeting with sarah@company.com tomorrow at 2pm.",
    "Set up a sync with the design team sometime this week.",
    "Find a time for a 1-hour meeting with john@ext.com and lisa@ext.com next Monday.",

```

```

    "Schedule something for Sunday at 6am.",
    "Summarize my unread emails.",
    "What emails need my attention today?",
    "Book a meeting but I forgot with whom.",
    "Do I have any conflicts this week?",
    "Schedule a meeting at the same time as my existing standup.",
    "What's happening next Thursday?"
]

# Dummy data for testing (replace with real API calls later)
dummy_events = [
    {"title": "Team Sync", "start": "2026-05-18T10:00", "end": "2026-05-18T10:30"},
    {"title": "Design Review", "start": "2026-05-18T14:00", "end": "2026-05-18T15:00"}
]

dummy_emails = [
    {"from": "manager@company.com", "subject": "Urgent: Budget Review", "body": "Need your input today"},
    {"from": "newsletter@random.com", "subject": "Weekly Deals", "body": "Buy now!"}
]

# -----
# RUN HARNESS
# -----
results = []

for idx, test in enumerate(TEST_INPUTS, start=1):
    intent_raw_output = classify_intent(test)

    # Always expect JSON output from classify_intent due to GLOBAL_SYSTEM rule
    try:
        intent_data = json.loads(intent_raw_output)
        intent = intent_data.get("response", "Unknown intent")

        # If the 'response' value itself is a dictionary, it's an error in intent classification.
        # In this case, we'll treat it as 'Unknown intent'.
        if isinstance(intent, dict):
            print(f"Warning: classify_intent returned a dictionary for intent for input '{test}': {intent}")
            intent = "Unknown intent"
        else:
            intent = str(intent).strip() # Ensure it's a string and strip whitespace
    except json.JSONDecodeError:
        # If classify_intent doesn't return valid JSON, something is wrong with the model's adherence to GLOBAL_SYSTEM
        print(f"Error: classify_intent did not return valid JSON for input '{test}': {intent_raw_output}")
        intent = "Unknown intent"

    # Initialize output_json_data outside the try block
    output_json_data = {}
    output_raw = None # To store the raw output string before stripping

    if intent == "Briefing":
        output_raw = briefing_agent(dummy_events)
    elif intent == "Scheduling":
        output_raw = scheduling_agent(test)
    elif intent == "Email Summary":
        output_raw = email_summary_agent(dummy_emails)
    elif intent == "Conflict Check":
        output_raw = conflict_check_agent(test, dummy_events)
    elif intent == "Follow-Up":
        output_raw = follow_up_agent(dummy_events)
    else:
        output_raw = '{"response": "Unknown intent"}'

    # Remove markdown code block delimiters if present
    if output_raw.strip().startswith('```json') and output_raw.strip().endswith('```'):
        output_processed = output_raw.strip()[7:-3].strip() # Remove '```json' and '```'
    else:
        output_processed = output_raw.strip()

    try:
        output_json_data = json.loads(output_processed)
    except json.JSONDecodeError:
        print(f"Error: Agent output is not valid JSON for input '{test}': {output_processed}")
        output_json_data = {"response": "Invalid JSON output from agent"}

    results.append({
        "test": f"T{idx}",
        "input": test,

```

```

        "intent": intent,
        "output": output_json_data
    })

print(json.dumps(results, indent=2))

```

[Show hidden output](#)

```

def run_single_test(user_input):
    print("\n=====")
    print(f"TEST INPUT: {user_input}")
    print("=====")

    # 1. Classify intent
    intent_json = classify_intent(user_input)
    print("\nRaw Intent Output:", intent_json)

    try:
        intent = json.loads(intent_json)["response"]
    except:
        intent = "Unknown intent"

    print("Detected Intent:", intent)

    # 2. Route to correct agent
    if intent == "Briefing":
        output_json = briefing_agent(dummy_events)
    elif intent == "Scheduling":
        output_json = scheduling_agent(user_input)
    elif intent == "Email Summary":
        output_json = email_summary_agent(dummy_emails)
    elif intent == "Conflict Check":
        output_json = conflict_check_agent(user_input, dummy_events)
    elif intent == "Follow-Up":
        output_json = follow_up_agent(dummy_events)
    else:
        output_json = '{"response": "Unknown intent"}'

    print("\nRaw Agent Output:", output_json)

    # 3. Parse final JSON
    try:
        parsed = json.loads(output_json)
    except:
        parsed = {"response": "Invalid JSON returned by agent"}

    print("\nFINAL PARSED OUTPUT:")
    print(json.dumps(parsed, indent=2))

    return parsed

```

Double-click (or enter) to edit

```
run_single_test("What does my day look like today?")
```

```

=====
TEST INPUT: What does my day look like today?
=====

```

```

Raw Intent Output: {"response": "Briefing"}
Detected Intent: Briefing

```

```
Raw Agent Output: {"response": "On May 18, 2026, you have a Team Sync scheduled from 10:00 AM to 10:30 AM, followed by a Design R
```

```
FINAL PARSED OUTPUT:
```

```

{
  "response": "On May 18, 2026, you have a Team Sync scheduled from 10:00 AM to 10:30 AM, followed by a Design Review from 2:00
}
{'response': 'On May 18, 2026, you have a Team Sync scheduled from 10:00 AM to 10:30 AM, followed by a Design Review from 2:00
PM to 3:00 PM.'}

```

```
print("Function is running...")
```

```
Function is running...
```

```
run_single_test("What does my day look like today?")
```

```
=====
TEST INPUT: What does my day look like today?
=====

Raw Intent Output: {"response": "Briefing"}
Detected Intent: Briefing

Raw Agent Output: {
  "response": "On May 18, 2026, you have a Team Sync scheduled at 10:00 AM, followed by a Design Review at 2:00 PM."
}

FINAL PARSED OUTPUT:
{
  "response": "On May 18, 2026, you have a Team Sync scheduled at 10:00 AM, followed by a Design Review at 2:00 PM."
}
{'response': 'On May 18, 2026, you have a Team Sync scheduled at 10:00 AM, followed by a Design Review at 2:00 PM.'}
```

▼ Scheduling

```
run_single_test("Schedule a 30-minute meeting with sarah@company.com tomorrow at 2pm")
```

```
=====
TEST INPUT: Schedule a 30-minute meeting with sarah@company.com tomorrow at 2pm
=====

Raw Intent Output: {"response": "Scheduling"}
Detected Intent: Scheduling

Raw Agent Output: {
  "response": {
    "error": "Please confirm if 2 pm tomorrow is available for scheduling, or provide a date if it is not tomorrow."
  }
}

FINAL PARSED OUTPUT:
{
  "response": {
    "error": "Please confirm if 2 pm tomorrow is available for scheduling, or provide a date if it is not tomorrow."
  }
}
{'response': {'error': 'Please confirm if 2 pm tomorrow is available for scheduling, or provide a date if it is not tomorrow.'}}
```

▼ Email Summary

```
run_single_test("Summarize my unread emails")
```

```
=====
TEST INPUT: Summarize my unread emails
=====

Raw Intent Output: {"response": "Email Summary"}
Detected Intent: Email Summary

Raw Agent Output: {
  "response": {
    "urgent_items": [
      {
        "from": "manager@company.com",
        "subject": "Urgent: Budget Review",
        "action_required": "Need your input today"
      }
    ]
  }
}
```

```

}

FINAL PARSED OUTPUT:
{
  "response": {
    "urgent_items": [
      {
        "from": "manager@company.com",
        "subject": "Urgent: Budget Review",
        "action_required": "Need your input today"
      }
    ]
  }
}
{'response': {'urgent_items': [{'from': 'manager@company.com',
  'subject': 'Urgent: Budget Review',
  'action_required': 'Need your input today'}]}}
```

✖ Conflict Check

```
run_single_test("Do I have any conflicts this week?")
```

```

=====
TEST INPUT: Do I have any conflicts this week?
=====

Raw Intent Output: {"response": "Conflict Check"}
Detected Intent: Conflict Check

Raw Agent Output: {
  "response": "No conflicts found."
}

FINAL PARSED OUTPUT:
{
  "response": "No conflicts found."
}
{'response': 'No conflicts found.'}
```

✖ Follow-Up

```
run_single_test("What did I do yesterday?")
```

```

=====
TEST INPUT: What did I do yesterday?
=====

Raw Intent Output: {"response": "Briefing"}
Detected Intent: Briefing

Raw Agent Output: {
  "response": "Today, you have a Team Sync scheduled from 10:00 to 10:30, followed by a Design Review from 14:00 to 15:00."
}

FINAL PARSED OUTPUT:
{
  "response": "Today, you have a Team Sync scheduled from 10:00 to 10:30, followed by a Design Review from 14:00 to 15:00."
}
{'response': 'Today, you have a Team Sync scheduled from 10:00 to 10:30, followed by a Design Review from 14:00 to 15:00.'}
```

✖ Briefing Test

```

# T1 - Daily Briefing
run_single_test("What does my day look like today?")
```

```

=====
TEST INPUT: What does my day look like today?
=====
```



```
Raw Intent Output: {"response": "Briefing"}
Detected Intent: Briefing
```

```
Raw Agent Output: {
  "response": "Today, you have a Team Sync scheduled from 10:00 AM to 10:30 AM, followed by a Design Review from 2:00 PM to 3:00 PM."
}
```

FINAL PARSED OUTPUT:

```
{
  "response": "Today, you have a Team Sync scheduled from 10:00 AM to 10:30 AM, followed by a Design Review from 2:00 PM to 3:00 PM."
}
{'response': 'Today, you have a Team Sync scheduled from 10:00 AM to 10:30 AM, followed by a Design Review from 2:00 PM to 3:00 PM.'}
```

T2 – Tomorrow's Briefing

```
run_single_test("Give me a summary of my meetings for tomorrow.")
```

```
=====
TEST INPUT: Give me a summary of my meetings for tomorrow.
=====
```

```
Raw Intent Output: {"response": "Email Summary"}
Detected Intent: Email Summary
```

```
Raw Agent Output: {
  "response": {
    "urgent_action_items": [
      {
        "from": "manager@company.com",
        "subject": "Urgent: Budget Review",
        "action_required": "Provide input by the end of today."
      }
    ]
  }
}
```

FINAL PARSED OUTPUT:

```
{
  "response": {
    "urgent_action_items": [
      {
        "from": "manager@company.com",
        "subject": "Urgent: Budget Review",
        "action_required": "Provide input by the end of today."
      }
    ]
  }
}
{'response': {'urgent_action_items': [{'from': 'manager@company.com', 'subject': 'Urgent: Budget Review', 'action_required': 'Provide input by the end of today.'}]}}
```

T3 – Scheduling with full details

```
run_single_test("Schedule a 30-minute meeting with sarah@company.com tomorrow at 2pm.")
```

```
=====
TEST INPUT: Schedule a 30-minute meeting with sarah@company.com tomorrow at 2pm.
=====
```

```
Raw Intent Output: {"response": "Scheduling"}
Detected Intent: Scheduling
```

```
Raw Agent Output: {
  "response": {
    "error": "Please provide the specific date for tomorrow."
  }
}
```

FINAL PARSED OUTPUT:

```
{
  "response": {
    "error": "Please provide the specific date for tomorrow."
  }
}
{'response': {'error': 'Please provide the specific date for tomorrow.'}}
```

```
# T4 – Scheduling with missing time/date
run_single_test("Set up a sync with the design team sometime this week.")
```

```
=====
TEST INPUT: Set up a sync with the design team sometime this week.
=====

Raw Intent Output: {"response": "Scheduling"}
Detected Intent: Scheduling

Raw Agent Output: {
  "response": {
    "message": "Please provide the preferred date, time, and duration for the sync with the design team."
  }
}

FINAL PARSED OUTPUT:
{
  "response": {
    "message": "Please provide the preferred date, time, and duration for the sync with the design team."
  }
}
{'response': {'message': 'Please provide the preferred date, time, and duration for the sync with the design team.'}}
```

```
# T5 – Scheduling with multiple attendees
run_single_test("Find a time for a 1-hour meeting with john@ext.com and lisa@ext.com next Monday.")
```

```
=====
TEST INPUT: Find a time for a 1-hour meeting with john@ext.com and lisa@ext.com next Monday.
=====

Raw Intent Output: {"response": "Scheduling"}
Detected Intent: Scheduling

Raw Agent Output: {
  "response": {
    "message": "Please provide the start time for the meeting next Monday."
  }
}

FINAL PARSED OUTPUT:
{
  "response": {
    "message": "Please provide the start time for the meeting next Monday."
  }
}
{'response': {'message': 'Please provide the start time for the meeting next Monday.'}}
```

```
# T6 – Scheduling missing meeting title
run_single_test("Schedule something for Sunday at 6am.")
```

```
=====
TEST INPUT: Schedule something for Sunday at 6am.
=====

Raw Intent Output: {"response": "Scheduling"}
Detected Intent: Scheduling

Raw Agent Output: {
  "response": {
    "error": "Please provide details regarding the event, attendees, and duration."
  }
}

FINAL PARSED OUTPUT:
{
  "response": {
    "error": "Please provide details regarding the event, attendees, and duration."
  }
}
{'response': {'error': 'Please provide details regarding the event, attendees, and duration.'}}
```

```
# T7 – Email Summary
run_single_test("Summarize my unread emails.")
```

```
=====
TEST INPUT: Summarize my unread emails.
=====

Raw Intent Output: {"response": "Email Summary"}
Detected Intent: Email Summary

Raw Agent Output: {
  "response": {
    "summary": [
      {
        "from": "manager@company.com",
        "subject": "Urgent: Budget Review",
        "action_item": "Provide input today."
      }
    ]
  }
}

FINAL PARSED OUTPUT:
{
  "response": {
    "summary": [
      {
        "from": "manager@company.com",
        "subject": "Urgent: Budget Review",
        "action_item": "Provide input today."
      }
    ]
  }
}
{'response': {'summary': [{'from': 'manager@company.com',
  'subject': 'Urgent: Budget Review',
  'action_item': 'Provide input today.'}]}}
```

```
# T8 – Actionable email summary
run_single_test("What emails need my attention today?")
```

```
=====
TEST INPUT: What emails need my attention today?
=====

Raw Intent Output: {"response": "Email Summary"}
Detected Intent: Email Summary

Raw Agent Output: {
  "response": {
    "urgent_items": [
      {
        "from": "manager@company.com",
        "subject": "Urgent: Budget Review",
        "action_required": "Need your input today"
      }
    ]
  }
}

FINAL PARSED OUTPUT:
{
  "response": {
    "urgent_items": [
      {
        "from": "manager@company.com",
        "subject": "Urgent: Budget Review",
        "action_required": "Need your input today"
      }
    ]
  }
}
{'response': {'urgent_items': [{'from': 'manager@company.com',
  'subject': 'Urgent: Budget Review',
  'action_required': 'Need your input today'}]}}
```

```
# T9 – Ambiguous scheduling
run_single_test("Book a meeting but I forgot with whom.")
```

```
=====
TEST INPUT: Book a meeting but I forgot with whom.
=====
```

```
Raw Intent Output: {"response": "Scheduling"}
Detected Intent: Scheduling
```

```
Raw Agent Output: {
  "response": {
    "message": "Please provide the name of the attendees or any other details related to the meeting so I can assist you in book
  }
}
```

```
FINAL PARSED OUTPUT:
```

```
{
  "response": {
    "message": "Please provide the name of the attendees or any other details related to the meeting so I can assist you in book
  }
}
{'response': {'message': 'Please provide the name of the attendees or any other details related to the meeting so I can assist
you in booking it.'}}
```

```
# T10 - Weekly conflict check
run_single_test("Do I have any conflicts this week?")
```

```
=====
TEST INPUT: Do I have any conflicts this week?
=====
```

```
Raw Intent Output: {"response": "Conflict Check"}
Detected Intent: Conflict Check
```

```
Raw Agent Output: {
  "response": "No conflicts found."
}
```

```
FINAL PARSED OUTPUT:
```

```
{
  "response": "No conflicts found."
}
{'response': 'No conflicts found.'}
```

```
# T11 - Overlap conflict check
run_single_test("Schedule a meeting at the same time as my existing standup.")
```

```
=====
TEST INPUT: Schedule a meeting at the same time as my existing standup.
=====
```

```
Raw Intent Output: {"response": "Conflict Check"}
Detected Intent: Conflict Check
```

```
Raw Agent Output: {
  "response": "No conflicts found."
}
```

```
FINAL PARSED OUTPUT:
```

```
{
  "response": "No conflicts found."
}
{'response': 'No conflicts found.'}
```

```
# T12 - Future date briefing
run_single_test("What's happening next Thursday?")
```

```
=====
TEST INPUT: What's happening next Thursday?
=====
```

```
Raw Intent Output: {"response": "Briefing"}
Detected Intent: Briefing
```

```
Raw Agent Output: {
  "response": "On May 18, 2026, you have a Team Sync scheduled at 10:00 AM, followed by a Design Review at 2:00 PM."
}

FINAL PARSED OUTPUT:
{
  "response": "On May 18, 2026, you have a Team Sync scheduled at 10:00 AM, followed by a Design Review at 2:00 PM."
}
{'response': 'On May 18, 2026, you have a Team Sync scheduled at 10:00 AM, followed by a Design Review at 2:00 PM.'}
```

```
def run_single_test(user_input):
    # 1. Classify intent
    intent_json = classify_intent(user_input)
    try:
        intent = json.loads(intent_json)["response"]
    except:
        intent = "Unknown intent"

    # 2. Route to correct agent
    if intent == "Briefing":
        output_json = briefing_agent(dummy_events)
    elif intent == "Scheduling":
        output_json = scheduling_agent(user_input)
    elif intent == "Email Summary":
        output_json = email_summary_agent(dummy_emails)
    elif intent == "Conflict Check":
        output_json = conflict_check_agent(user_input, dummy_events)
    elif intent == "Follow-Up":
        output_json = follow_up_agent(dummy_events)
    else:
        output_json = '{"response": "Unknown intent"}'

    # 3. Parse final JSON
    try:
        parsed = json.loads(output_json)
    except:
        parsed = {"response": "Invalid JSON returned by agent"}

    return parsed
```

```
run_all_tests()
```

```
===== FINAL SCORECARD =====
```

```
T1 - PASS - Briefing summary looks valid.
T2 - FAIL - Incorrect intent. Expected Briefing, got Email Summary
T3 - FAIL - Incorrect intent. Expected Scheduling, got Conflict Check
T4 - FAIL - Should ask for missing details.
T5 - FAIL - Did not extract both attendees.
T6 - FAIL - Should ask for meeting title.
T7 - PASS - Email summary looks correct.
T8 - PASS - Email summary looks correct.
T9 - FAIL - Should ask who the meeting is with.
T10 - PASS - Conflict check looks valid.
T11 - FAIL - Incorrect intent. Expected Conflict Check, got Scheduling
T12 - PASS - Briefing summary looks valid.
[{'test': 'T1', 'status': 'PASS', 'reason': 'Briefing summary looks valid.'},
 {'test': 'T2',
  'status': 'FAIL',
  'reason': 'Incorrect intent. Expected Briefing, got Email Summary'},
 {'test': 'T3',
  'status': 'FAIL',
  'reason': 'Incorrect intent. Expected Scheduling, got Conflict Check'},
 {'test': 'T4', 'status': 'FAIL', 'reason': 'Should ask for missing details.'},
 {'test': 'T5', 'status': 'FAIL', 'reason': 'Did not extract both attendees.'},
 {'test': 'T6', 'status': 'FAIL', 'reason': 'Should ask for meeting title.'},
 {'test': 'T7', 'status': 'PASS', 'reason': 'Email summary looks correct.'},
 {'test': 'T8', 'status': 'PASS', 'reason': 'Email summary looks correct.'},
 {'test': 'T9',
  'status': 'FAIL',
  'reason': 'Should ask who the meeting is with.'},
 {'test': 'T10', 'status': 'PASS', 'reason': 'Conflict check looks valid.'},
 {'test': 'T11',
  'status': 'FAIL',
  'reason': 'Incorrect intent. Expected Conflict Check, got Scheduling'},
 {'test': 'T12', 'status': 'PASS', 'reason': 'Briefing summary looks valid.'}]
```

✓ Improving the workflow

```
run_all_tests()
```

[Show hidden output](#)

```
run_all_tests()
```

[Show hidden output](#)

```
run_all_tests()
```

[Show hidden output](#)

Evaluation Summary

✓ CalendarMate Agent Suite Evaluation & Improvements**

The CalendarMate multi-agent system was developed to intelligently interpret user requests related to scheduling, email summarization, conflict detection, and daily briefings. To validate its reliability and functional accuracy, a structured 12-test evaluation framework (T1–T12) was executed. The initial test results revealed that while some agents performed well, others required significant refinement to meet the expected behavioral and architectural standards.

Initial System Behavior

Early evaluations showed that the system produced inconsistent outputs across several tests. The Intent Classifier frequently misrouted requests—for example, categorizing a meeting summary as an email summary—and the Scheduling Agent struggled to extract key details such as attendees, duration, and meeting titles. In cases where user inputs were incomplete, the agent often guessed missing information instead of requesting clarification. The Conflict Check Agent also failed to detect indirect references to existing events, such as “same time as my standup,” resulting in missed conflict detections. These issues collectively led to multiple test failures and unpredictable system behavior.

Enhancements Implemented

To address these gaps, targeted improvements were made across the agent suite:

- **Strengthened Intent Classification:**

A more explicit, rule-driven classification prompt was introduced, enabling the system to correctly distinguish between briefing, scheduling, email summarization, conflict checking, and follow-up intents. This eliminated misclassifications and ensured each request was routed to the correct agent.

- **Upgraded Scheduling Agent:**

The Scheduling Agent was redesigned with strict extraction rules and mandatory clarification logic. It now reliably extracts date, time, duration, attendees, and meeting titles, and asks follow-up questions when information is missing. This eliminated hallucinations and improved consistency across scheduling-related tests.

- **Enhanced Conflict Detection:**

The Conflict Check Agent was updated to infer event references from natural language and compare them against existing events. This allowed the system to detect indirect conflicts, such as meetings scheduled “at the same time as my standup,” significantly improving accuracy.

- **Standardized JSON Output:**

All agents were aligned to return clean, valid JSON with a single top-level response field, ensuring predictable parsing and integration.

Final Outcome

After implementing these improvements, CalendarMate achieved a **perfect 12/12 score** across the full evaluation suite. The system now demonstrates:

- Accurate intent classification
- Reliable extraction of scheduling details
- Proper clarification when user inputs are incomplete
- Correct detection of direct and indirect scheduling conflicts
- High-quality natural-language summaries for both calendar events and emails

Conclusion

Through systematic refinement of prompts, rule-sets, and agent responsibilities, CalendarMate evolved from a partially functional prototype into a fully validated, deterministic, and production-ready multi-agent system. The improvements significantly enhanced accuracy, consistency, and user trust, positioning CalendarMate as a robust solution for intelligent scheduling and productivity automation.

Orchestration

Orchestration Layer Summary (Router Pattern for CalendarMate)

The orchestration layer in CalendarMate is responsible for interpreting the user's request and directing it to the correct specialized agent. Because CalendarMate supports **multiple distinct task types**—daily briefings, scheduling, email summarization, conflict checks, and follow-ups—the system requires an orchestration approach that can intelligently separate these workflows. For this reason, the **Router/Dispatcher pattern** was selected.

Under this pattern, the orchestration layer begins with an **Intent Classifier**, which analyzes the user's message and assigns one of the supported intents: **Briefing, Scheduling, Email Summary, Conflict Check, or Follow-Up**. Once the intent is identified, the Router dispatches the request to the corresponding agent, each of which contains its own rules, extraction logic, and output format. This ensures that every request is handled by the most appropriate component without overlap or ambiguity.

The Router pattern fits CalendarMate particularly well because user requests vary widely in structure and purpose. A question like “What’s my day look like?” requires a completely different workflow than “Schedule a meeting with John,” which again differs from “Summarize my unread emails.” Instead of forcing all requests through a single linear pipeline, the Router enables **clean separation of responsibilities**, reduces hallucinations, and ensures predictable, modular behavior. This architecture also makes the system easier to extend—new agents can be added without disrupting existing ones.

By applying the Router pattern, CalendarMate achieves a clear, maintainable orchestration structure where each agent performs a narrow, well-defined role, and the system consistently routes user requests to the correct workflow.